



UNIVERSIDAD DE
COSTA RICA
Sede del Atlántico

RP Recinto de
Paraíso

UNIVERSIDAD DE COSTA RICA
SEDE DEL ATLÁNTICO - RECINTO PARAÍSO
CARRERA DE INFORMÁTICA EMPRESARIAL
CURSO: IF0009 - Desarrollo de Software IV
PROFESOR: Mag. Jonathan Granados C.
SEMESTRE: II-2026

Tema 3.1: Capa de Datos y Persistencia Empresarial con Spring Data JPA

Resumen

Esta lectura conceptual aborda los fundamentos de la capa de datos en aplicaciones Java empresariales utilizando **Spring Data JPA** e **Hibernate** como motor de persistencia. El objetivo de este tema es guiar al estudiantado para comprender cómo realizar el mapeo objeto-relacional (ORM), implementar repositorios para operaciones CRUD, consumir procedimientos almacenados y aplicar paginación física y ordenamiento a nivel de base de datos para garantizar el óptimo rendimiento y escalabilidad de los sistemas de software.

Concepto Principal (El 'Qué')

1. El Mapeo Objeto-Relacional (ORM)

En el desarrollo de software orientado a objetos, los datos se representan como redes de objetos interconectados a través de referencias. Por otro lado, las bases de datos relacionales organizan la información en tablas de dos dimensiones formadas por filas y columnas vinculadas mediante llaves foráneas. Esta discrepancia conceptual se denomina **desajuste de impedancia objeto-relacional**.

Un **Mapeo Objeto-Relacional (ORM)** es un patrón de diseño y una herramienta de software que automatiza la traducción bidireccional entre los objetos de la memoria de la aplicación y las tuplas del modelo relacional. En el ecosistema Java, **JPA (Java Persistence API)** es la especificación estándar que define las anotaciones y reglas de este mapeo, mientras que **Hibernate** es el proveedor de persistencia (el motor de software que implementa las reglas de JPA).

2. Entidades JPA

Una **Entidad JPA** es una clase Java estándar (POJO) que ha sido decorada con metadatos (anotaciones) para asociarla directamente con una tabla de la base de datos relacional. Las principales anotaciones de mapeo son: * **@Entity**: Indica que la clase es una entidad persistente y será gestionada por el contexto de persistencia de JPA. * **@Table**: Especifica el nombre exacto de la tabla correspondiente en la base de datos relacional. * **@Id**: Define la propiedad que actúa como llave primaria de la entidad. * **@GeneratedValue**: Declara la estrategia de generación automática para la llave primaria (ej. incremento automático por base de datos). * **@Column**: Asocia una propiedad a una columna específica, permitiendo definir restricciones de longitud, valores nulos y nombres personalizados.

3. Repositorios de Spring Data JPA

Tradicionalmente, realizar operaciones en base de datos requería escribir código boilerplate extenso (patrón DAO) implementando sentencias SQL y mapeando manualmente los resultados. **Spring Data JPA** introduce la abstracción de repositorios, permitiendo declarar interfaces que extienden de **JpaRepository**.

Al hacer esto, el framework genera dinámicamente en tiempo de ejecución la implementación de métodos estándar de manipulación de datos (CRUD), así como la traducción de métodos de consulta estructurados según el nombre del método (ej. **findByActivo**).

4. Consumo de Procedimientos Almacenados

En sistemas corporativos, es frecuente delegar lógica compleja o procesamiento masivo a **procedimientos almacenados** en el motor de base de datos. JPA proporciona soporte para

consumir estos procedimientos mediante las anotaciones `@NamedStoredProcedureQuery` y `@StoredProcedureParameter`, los cuales definen los parámetros de entrada (IN) y salida (OUT) del procedimiento almacenado para ser invocados de forma tipada mediante métodos del repositorio decorados con la anotación `@Procedure`.

5. Paginación y Ordenamiento Físico

Consultar tablas con millones de registros y cargarlos completos en la memoria RAM de la aplicación es uno de los fallos de diseño más graves en desarrollo de software, pues colapsa el servidor y degrada el rendimiento de la red.

La **paginación física** consiste en solicitar al motor de base de datos que retorne únicamente un subconjunto limitado de filas correspondiente a una página específica (ej. mediante cláusulas `LIMIT` y `OFFSET` en SQL de forma transparente). Spring Data JPA simplifica esto a través de las interfaces `Pageable` (que contiene la página solicitada, el tamaño y el criterio de ordenamiento `Sort`) y `Page` (que contiene los registros retornados y metadatos adicionales, como el número total de páginas y elementos totales).

Analogía (La Intuición)

- **El ORM como Traductor Simultáneo:** Imagine una conferencia internacional donde el ponente solo habla español (Java y Objetos) y el público solo entiende mandarín (SQL y Tablas Relacionales). Sin un traductor, la comunicación es imposible. El ORM actúa como el traductor simultáneo en cabina: traduce las ideas complejas (objetos) en sentencias estructuradas en mandarín (SQL) en tiempo real, permitiendo que ambos lados interactúen de forma fluida sin tener que aprender el lenguaje del otro.
- **La Paginación como Búsqueda en una Enciclopedia:** Imagine que usted necesita leer el tomo 3 de una enciclopedia de 20 tomos. Sería absurdo obligar al bibliotecario a cargar los 20 tomos juntos sobre su mesa de lectura (cargar toda la tabla en memoria RAM), sobrecargando su espacio de estudio físico. En su lugar, usted le solicita específicamente el volumen 3, ocupando únicamente el espacio mínimo necesario y devolviendo los demás volúmenes al estante. La paginación física hace exactamente eso con los registros de la base de datos.

Ejemplo de Código e Implementación (El 'Cómo')

A continuación se muestra una implementación completa de la capa de datos utilizando Spring Data JPA para gestionar la entidad `Estudiante`, incorporando llamadas a procedimientos almacenados y consultas paginadas.

1. Entidad Relacional (`Estudiante.java`)

Cree la entidad y defina el mapeo de base de datos junto con el procedimiento almacenado para calcular el promedio de notas en la ruta:

`src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica2/domain/Estudiante.java`

```
package cr.ac.ucr.paraiso.ie.carnet.practica2.domain;

import jakarta.persistence.*;

@Entity
@Table(name = "estudiantes")
@NamedStoredProcedureQuery(
    name = "Estudiante.obtenerPromedio",
    procedureName = "SP_OBTENER_PROMEDIO",
    parameters = {
        @StoredProcedureParameter(
            mode = ParameterMode.IN,
            name = "p_estudiante_id",
            type = Integer.class
        ),
        @StoredProcedureParameter(
            mode = ParameterMode.OUT,
            name = "p_promedio",
            type = Double.class
        )
    }
)
public class Estudiante {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id_estudiante")
    private Integer id;

    @Column(name = "nombre_completo", length = 100, nullable = false)
    private String nombre;
```

```

    @Column(name = "carnet_universitario", length = 10, unique = true)
    private String carnet;

    @Column(name = "estado_activo")
    private boolean activo;

    public Estudiante() {}

    public Estudiante(String nombre, String carnet, boolean activo) {
        this.nombre = nombre;
        this.carnet = carnet;
        this.activo = activo;
    }

    public Integer getId() { return id; }
    public void setId(Integer id) { this.id = id; }

    public String getNombre() { return nombre; }
    public void setNombre(String nombre) { this.nombre = nombre; }

    public String getCarnet() { return carnet; }
    public void setCarnet(String carnet) { this.carnet = carnet; }

    public boolean isActive() { return activo; }
    public void setActive(boolean activo) { this.activo = activo; }
}

```

2. Repositorio (EstudianteRepository.java)

Cree la interfaz del repositorio y declare la firma del procedimiento almacenado y el método paginado en:

src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica2/data/EstudianteRepository.java

```

package cr.ac.ucr.paraiso.ie.carnet.practica2.data;

import cr.ac.ucr.paraiso.ie.carnet.practica2.domain.Estudiante;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.Pageable;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.query.Procedure;
import org.springframework.data.repository.query.Param;
import org.springframework.stereotype.Repository;

```

```

@Repository
public interface EstudianteRepository
    extends JpaRepository<Estudiante, Integer> {

    // Invocar procedimiento almacenado mediante su nombre configurado
    @Procedure(name = "Estudiante.obtenerPromedio")
    Double obtenerPromedioEstudiante(
        @Param("p_estudiante_id") Integer estudianteId
    );

    // Consulta que filtra por estado activo y aplica paginación
    Page<Estudiante> findByActivo(
        boolean activo,
        Pageable pageable
    );
}

```

3. Servicio de Lógica de Negocio (EstudianteService.java)

Implemente la clase de servicio para encapsular las llamadas transaccionales al repositorio en la ruta:

src/main/java/cr/ac/ucr/paraiso/ie/carnet/practica2/business/EstudianteService.java

```

package cr.ac.ucr.paraiso.ie.carnet.practica2.business;

import cr.ac.ucr.paraiso.ie.carnet.practica2.data.EstudianteRepository;
import cr.ac.ucr.paraiso.ie.carnet.practica2.domain.Estudiante;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.data.domain.Page;
import org.springframework.data.domain.PageRequest;
import org.springframework.data.domain.Pageable;
import org.springframework.data.domain.Sort;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

@Service
public class EstudianteService {

    @Autowired
    private EstudianteRepository repository;

    @Transactional
    public Estudiante guardar(Estudiante estudiante) {
        return repository.save(estudiante);
    }
}

```

```

    }
}

@Transactional(readOnly = true)
public Page<Estudiante> listarActivosPaginados(
    int pagina,
    int tamano
) {
    // Configurar paginación indexada en 0 y ordenar por nombre
    Pageable controlPaginacion = PageRequest.of(
        pagina,
        tamano,
        Sort.by("nombre").ascending()
    );
    return repository.findByActivo(true, controlPaginacion);
}

@Transactional
public Double obtenerPromedioNotas(Integer idEstudiante) {
    return repository.obtenerPromedioEstudiante(idEstudiante);
}
}

```

Explicación del Código Línea por Línea

- **Estudiante.java (Línea 6-21):** Decoramos la entidad con `@NamedStoredProcedureQuery`, asociándola a un procedimiento de base de datos llamado `SP_OBTENER_PROMEDIO` con un parámetro de tipo entero `p_estudiante_id` como entrada (IN) y una variable decimal `p_promedio` como salida (OUT).
- **Estudiante.java (Línea 26-30):** La anotación `@GeneratedValue(strategy = GenerationType.IDENTITY)` delega al motor de base de datos (como MySQL o PostgreSQL) la responsabilidad de generar identificadores únicos consecutivos al insertar un nuevo registro.
- **EstudianteRepository.java (Línea 15-18):** Declaramos el método `@Procedure(name = "Estudiante.obtenerPromedio")`. Al invocar este método desde Java, Spring Data ejecuta la consulta de procedimiento tipada y retorna el valor de salida (OUT) mapeado como un objeto `Double`.
- **EstudianteRepository.java (Línea 21-24):** El método `findByActivo` recibe un parámetro `Pageable`. Spring Data JPA traduce automáticamente esto a nivel de base de datos añadiendo la ordenación y límites de filas a la consulta SQL nativa.

- **EstudianteService.java (Línea 26-34):** En el método `listarActivosPaginados`, construimos un objeto `PageRequest.of` pasando el índice de página (comenzando en 0), la cantidad de elementos por página, y un objeto `Sort` indicando la ordenación alfabética ascendente por la propiedad `nombre`.

Trampas Comunes

El Problema de Consultas N+1: Al mapear relaciones (como `@ManyToOne`) con carga ansiosa (`FetchType.EAGER`), Hibernate realiza una consulta inicial para obtener la entidad y luego N consultas adicionales para obtener los registros asociados de cada elemento de la lista.

- *Solución:* Configure las relaciones por defecto como `FetchType.LAZY` (carga perezosa) y utilice consultas con sentencias `JOIN FETCH` cuando requiera recuperar los datos de manera combinada.

Excepciones de Lazy Initialization (`LazyInitializationException`): Ocurre cuando se intenta acceder a una propiedad de carga perezosa (`LAZY`) fuera del contexto de una transacción activa (sin la anotación `@Transactional`), ya que la sesión de Hibernate con la base de datos ha sido cerrada.

- *Solución:* Asegúrese de que el acceso a colecciones perezosas ocurra dentro de métodos de servicio anotados con `@Transactional`.

Falta de Transaccionalidad en Procedimientos Almacenados: Invocaciones a procedimientos almacenados en base de datos que realicen modificaciones de datos fallarán si el método de ejecución en Java no está envuelto en una transacción.

- *Solución:* Decore el método de llamada del servicio o repositorio con la anotación `@Transactional`.

Pregunta de Reflexión

Si una aplicación web empresarial con 500.000 usuarios activos realiza una búsqueda de transacciones y las carga en una lista de memoria en el servidor (`List<Transaccion>`) para que el frontend realice la paginación de forma visual: *

- ¿Qué impacto tendrá este diseño a nivel de consumo de memoria RAM, carga en la base de datos y rendimiento de red?
- * ¿Cómo soluciona Spring Data JPA este escenario mediante la interfaz Pageable?